

AI Based Alert Prioritization for Network Intrusion Detection

Using Machine Learning

CSCI 5742: Cybersecurity Programming and Analytics
Spring 2026

Joseph Tesoriero

University of Colorado Denver

`joseph.tesoriero@ucdenver.edu`

Tejal Jadhav

University of Colorado Denver

`tejalvilas.jadhav@ucdenver.edu`

Abstract

Intrusion detection systems (IDS) generate a large number of alerts, many of which are false positives or low-priority events. Although machine learning models have improved the detection of malicious network traffic, most existing approaches focus primarily on classification accuracy rather than on how alerts should be prioritized after detection. In practical environments, a high probability score does not always reflect the true impact or risk associated with an attack.

This project proposes a machine learning-based alert prioritization framework built on top of a standard intrusion detection model. The system first classifies network traffic as benign or malicious and produces a probability score. A risk scoring layer is then applied, combining detection confidence with contextual factors such as attack severity and asset importance. Alerts are ranked according to this combined risk score to better reflect their overall significance.

The proposed framework will be evaluated using a publicly available intrusion detection dataset. Performance will be measured using traditional classification metrics along with ranking-based metrics such as Precision@K and Recall@K. The objective is to demonstrate that incorporating contextual risk factors improves alert prioritization compared to probability-based ranking alone.

Contents

1	Introduction	4
1.1	Problem Statement	4
1.2	Motivation and Significance	4
2	Threat Model and Assumptions	5
2.1	Adversary Capabilities	5
2.2	System Assumptions	6
3	Research Objectives and Questions	6
4	Related Work	7
4.1	Overview of Existing Approaches	7
4.2	Comparative Summary	8
4.3	Identified Research Gaps	8
5	Proposed Approach and System Design	9
5.1	High-Level Architecture	9
5.2	Core Components	10
5.3	Implementation Plan	11
6	Methodology and Evaluation Plan	11
6.1	Experimental Setup	11
6.2	Datasets or Traffic Sources	12
6.3	Evaluation Metrics	12
6.4	Baselines and Comparisons	13
7	Expected Contributions	13
8	Risks and Contingency Plan	14
9	Work Plan and Timeline	15
10	Ethical Considerations	15
11	Conclusion	15

1 Introduction

1.1 Problem Statement

Enterprise networks rely on monitoring systems such as Intrusion Detection Systems (IDS) and Security Information and Event Management (SIEM) platforms to detect suspicious or malicious activity. These systems continuously analyze network traffic and generate alerts whenever abnormal behavior is identified. While this improves visibility into potential threats, it also creates a major challenge: a very large number of alerts are produced daily. Research shows that organizations may receive thousands of alerts per day, many of which are false positives, leading to alert fatigue and delayed response to critical incidents [1]. Prior work also highlights that practitioners often struggle to manage this alert volume effectively [1].

In this project, we consider a realistic network environment that includes web servers, database servers, user endpoints, and internal infrastructure systems. These assets may store sensitive information or support important services. The attacker goals addressed include common network-based threats such as brute force authentication attempts, denial-of-service (DoS/DDoS) attacks, web application exploitation (e.g., SQL injection and cross-site scripting), infiltration, botnet activity, and port scanning. These attack categories align with those represented in the CICIDS2017 dataset introduced by Sharafaldin et al. [2], which was designed to reflect realistic network traffic and diverse attack scenarios. The dataset also addresses limitations of earlier IDS datasets, including outdated attack types and limited feature diversity [2].

Most machine learning-based IDS approaches focus on classification. They are trained to label traffic as benign or malicious and are evaluated using metrics such as precision, recall, and F1-score [2]. While these metrics measure detection accuracy, they do not determine how urgent or impactful an alert is in context. A model may assign a high probability to a low-impact event and a slightly lower probability to a high-impact attack targeting a critical asset. Therefore, classification confidence alone does not fully represent operational risk.

Recent research has explored automated alert triage and prioritization frameworks to reduce alert overload [3, 4]. However, many approaches focus on predicting analyst actions or alert actionability rather than defining a structured and interpretable way to combine detection confidence with contextual factors such as attack severity and asset importance.

The specific cybersecurity problem addressed in this project is the lack of a structured, context-aware alert prioritization mechanism that bridges detection output with practical operational risk. This work proposes adding a transparent prioritization layer on top of a machine learning detection model. The system computes a risk score for each alert by combining prediction confidence with contextual attributes such as attack severity and asset importance. Alerts are then ranked based on this score to better reflect their potential impact within the network.

1.2 Motivation and Significance

The alert overload problem has become a well-recognized challenge in cybersecurity operations. Organizations generate a high volume of security alerts daily, many of which are low-risk or false positives [1]. This situation, commonly referred to as alert fatigue, reduces efficiency and increases the possibility that

serious threats may not be addressed in time [1]. Even though machine learning has improved detection accuracy, the number of generated alerts remains large, and not all malicious alerts carry the same level of impact.

Detection and prioritization are different tasks. Classification models determine whether traffic is malicious, but they do not decide how urgent or damaging an alert may be in practice. For example, two alerts may both be classified as malicious, yet one may target a critical database server while the other targets a low-value internal machine. Treating both alerts equally may lead to inefficient use of response resources. Recent work on alert triage has emphasized the need to move beyond pure detection toward more context-aware prioritization [3, 4].

From a research perspective, much of the IDS literature focuses on improving classification performance using benchmark datasets such as CICIDS2017 [2]. Evaluation typically centers on precision, recall, and F1-score. While these metrics are essential for measuring detection capability, they do not address how alerts should be ranked after detection. This creates a gap between academic detection research and practical operational decision-making.

The intended contribution of this project is the design and implementation of a structured and interpretable alert prioritization framework built on top of an existing machine learning detection model. The proposed system introduces a risk scoring mechanism that combines model confidence with contextual factors such as attack severity and asset importance. By ranking alerts according to this composite score, the framework aims to provide a more practical and meaningful way to prioritize security alerts in enterprise environments.

2 Threat Model and Assumptions

This project focuses on improving alert prioritization after an intrusion detection system (IDS) has already generated alerts. The goal is not to design new attack techniques or defensive mechanisms, but to evaluate how alerts can be ranked more effectively using machine learning confidence and contextual information. Therefore, the threat model defines the type of attackers, the environment considered, and the boundaries of what the system is expected to handle.

We consider a realistic network environment consisting of servers, endpoints, and internal systems that generate network traffic logs. The system processes alerts generated by an IDS and computes a risk score for prioritization. The evaluation is conducted using labeled datasets such as CICIDS2017, which contain both benign and malicious traffic scenarios. Experiments are restricted to controlled and offline environments using publicly available datasets. No live network scanning or active exploitation is performed.

This project is prioritization-focused rather than prevention-focused. The IDS detection layer is assumed to already exist and generate alerts. Our contribution operates after detection, in the alert ranking phase.

2.1 Adversary Capabilities

We assume an attacker who can send malicious traffic over the network. The attacker may attempt common attacks such as brute force login attempts, denial-of-service (DoS/DDoS), SQL injection, cross-

site scripting, infiltration, botnet communication, or port scanning. These attacks are consistent with those included in the CICIDS2017 dataset.

The attacker is assumed to have network-level access but does not initially have full control over the system. The attacker may generate repeated or automated traffic that causes multiple alerts to be triggered. We do not assume highly advanced evasion techniques specifically targeting the prioritization model.

The attack surface considered in this project is limited to network traffic monitored by the IDS. Physical attacks, insider misuse without network traces, and attacks outside monitored traffic are out of scope.

2.2 System Assumptions

We assume that an IDS or monitoring system is already deployed and generates alerts with associated features. The detection model (for example, Random Forest or XGBoost) produces a probability score indicating how likely the traffic is malicious.

We assume that some contextual information is available, such as the type of attack and the importance of the targeted asset (e.g., database server vs. user endpoint). Asset importance values can be predefined based on system role.

The prioritization layer works after detection and does not modify the detection model itself. The evaluation is dataset-based and offline. Real-time deployment challenges, large-scale distributed infrastructure, and production-level system constraints are not considered in this project.

3 Research Objectives and Questions

The primary objective of our project is to design and evaluate an AI based framework for prioritizing intrusion detection system (IDS) alerts using machine learning techniques. Modern IDS generate a high volume of alerts, many of which can be false positives or low priority. This creates an operational cost for analysts and can increase the likelihood that a critical threat is overlooked or delayed. The research aims to address these challenges by improving both classification accuracy and alert ranking effectiveness.

Objective 1

Develop and implement machine learning based alert prioritization system that classifies and ranks IDS alerts using features from the CIC-IDS2017 dataset. The system includes baseline approach (FIFO alert processing, threshold/probability based alert processing, and logistic regression) as well as ensemble models like Random Forest and XGBoost. In addition to the network traffic classification, a risk score will be designed to rank alerts using a contextual weighting.

Success will be measured using standard metrics (Accuracy, Precision, Recall, F1-score, ROC-AUC) and ranking based metrics (Precision@k and Recall@K). We will be looking to minimize false positive rates to improve the proportion of true attacks within the top ranked alerts.

Objective 2

Evaluate the feasibility and impact of the proposed system. This includes measuring detection latency, scalability, and comparing probability only ranking with risk adjusted rankings. The objective is to ensure improvements in detection performance translate to meaningful gains in real world environments.

Success will be judged by measurable improvements over baseline methods/models in both detection accuracy and ranking effectiveness, while maintaining and acceptable latency for real time analysis.

Research Questions

- **RQ1:** Does an ensemble learning model (Random Forest with XGBoost) improve detection performance compared to a linear model such as logistic regression?
- **RQ2:** Does a risk adjusted ranking improve Precision@K and Recall@K compared to a probability only ranking?
- **RQ3:** How effectively does our proposed system reduce false positives relative to threshold based or FIFO alert handling?
- **RQ4:** What is the detection latency of our model, and if the latency suitable for real time intrusion detection?

4 Related Work

4.1 Overview of Existing Approaches

Most research in intrusion detection focuses on improving classification performance. Using publicly available datasets such as CICIDS2017, many studies apply machine learning models including Logistic Regression, Random Forest, Support Vector Machines, and Gradient Boosting techniques [2]. These approaches are typically evaluated using classification metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. While these metrics measure how well malicious traffic can be detected, they do not address how alerts should be prioritized once multiple malicious events are identified.

In parallel, research in security operations centers (SOCs) has examined the challenge of alert overload. Survey-based studies emphasize that effective prioritization requires contextual information, such as attack severity, asset importance, frequency of occurrence, and analyst feedback [1]. However, there is no widely adopted or standardized method for combining these contextual factors into a clear and interpretable prioritization framework.

More recent machine learning-based triage systems attempt to improve prioritization by predicting alert actionability or escalation likelihood [3, 4]. These systems aim to learn from historical analyst decisions and automate aspects of alert handling. Although promising, such approaches often depend on proprietary SOC data or labeled escalation outcomes, which may not be available in publicly accessible intrusion detection datasets.

Overall, prior work tends to focus either on improving detection accuracy or on modeling analyst decision processes. Fewer approaches explicitly design a simple, transparent risk scoring mechanism that directly combines detection confidence with contextual severity in a way that can be evaluated using standard intrusion detection datasets such as CICIDS2017.

4.2 Comparative Summary

Study	Approach	Dataset	Limitation
Sharafaldin et al. [2]	ML-based IDS / Dataset Design	CICIDS2017	Focuses on detection accuracy; does not address alert prioritization
Jalalvand et al. [1]	Alert prioritization survey	SOC environments	Identifies prioritization factors but does not propose a concrete scoring framework
Turcotte et al. (AACT) [3]	ML-based alert classification and triage	Operational SOC alert data	Requires historical analyst decision labels; limited reproducibility with public datasets
Gelman et al. [4]	ML-based escalation prediction	Operational alert handling data	Focuses on analyst action prediction rather than transparent risk scoring

Table 1: Comparison of prior work

4.3 Identified Research Gaps

Based on the review of prior work, several important gaps remain in the literature. First, a large portion of intrusion detection research focuses on improving classification accuracy using metrics such as precision, recall, F1-score, and ROC-AUC. While these studies demonstrate strong detection performance, they do not address how alerts should be prioritized after classification. A high probability of malicious activity does not necessarily indicate that an alert has the highest operational impact.

Second, research in Security Operations Centers emphasizes the importance of contextual information such as attack severity, asset importance, and operational relevance when prioritizing alerts. However, many of these approaches are either descriptive in nature or rely on proprietary SOC data and historical analyst decision labels. As a result, they are difficult to reproduce or evaluate using publicly available intrusion detection datasets.

Third, recent machine learning based alert triage systems often attempt to predict analyst behavior or escalation outcomes. Although effective in operational environments, these systems primarily model human decision-making rather than defining a transparent and interpretable risk scoring framework that can be

independently evaluated.

Overall, there is a clear gap between detection-focused research and practical, context aware alert prioritization methods that can be evaluated using standard datasets such as CICIDS2017. Few studies explicitly design a structured risk scoring layer that combines detection confidence with contextual severity factors in a reproducible and interpretable manner.

This project addresses this gap by proposing a transparent risk scoring framework that integrates model probability with contextual attributes and evaluates its effectiveness using ranking based metrics.

5 Proposed Approach and System Design

5.1 High-Level Architecture

The proposed system follows a layered architecture that separates detection from prioritization. Network traffic data is first processed and passed to a machine learning detection model, such as Random Forest or XGBoost. The detection model classifies each traffic instance as benign or malicious and produces a probability score representing its confidence.

Instead of using this probability directly for decision-making, the system introduces an additional risk scoring layer. This layer combines the model’s prediction confidence with contextual factors such as attack severity and asset importance to compute a final risk score. The purpose of this layer is to better reflect the real-world impact of an alert rather than relying only on classification accuracy.

Finally, alerts are ranked based on their computed risk score and presented through a structured dashboard or output interface. This layered design ensures modularity, allowing the prioritization mechanism to operate independently of the detection model while improving the practical usefulness of generated alerts.

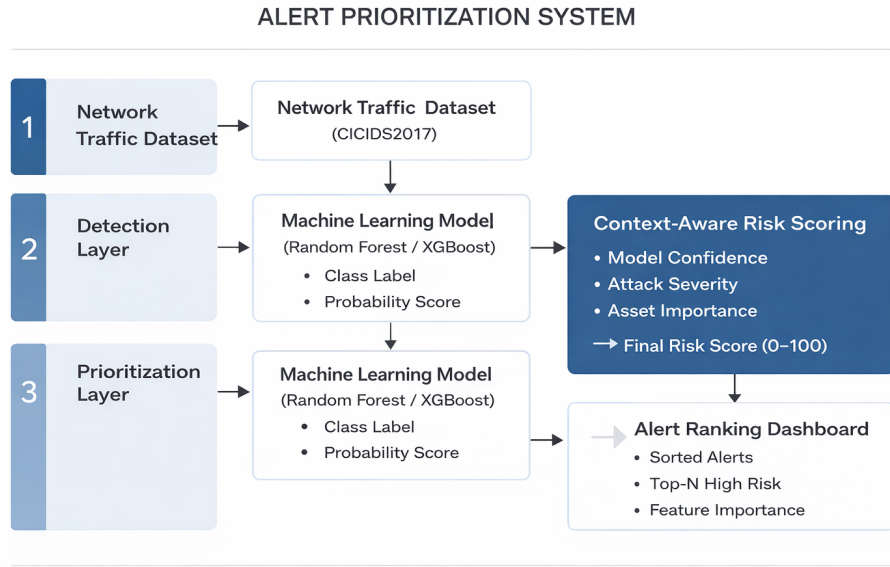


Figure 1: High-level system architecture

5.2 Core Components

Our system has four main parts, and each one does a specific job.

1. **Data Processing module:** This part prepares the data before training the model. It cleans the dataset, selects useful features, and splits the data into training and testing sets. We use a labeled dataset like CICIDS2017 so the model can learn the difference between normal and malicious traffic.
2. **Detection module:** This is the machine learning model, such as Random Forest or XGBoost. Its job is to classify traffic as benign or malicious. It also gives a probability score, which tells us how confident the model is about its prediction. This probability becomes the base risk value.
3. **Risk Scoring module:** This module is the main contribution of this project. Instead of using only the model's probability, this module combines three things: model confidence, attack severity (how dangerous the attack type is), and asset importance (how important the targeted system is). These are combined into a final risk score between 0 and 100. This score reflects how serious the alert is in practice.
4. **Alert Ranking and Visualization module:** This sorts all alerts based on the final risk score. The highest-risk alerts appear at the top. The results can be shown in a simple dashboard or table so that high-priority alerts are easier to identify.

Together, these components create a system where detection happens first, and then prioritization improves decision-making.

5.3 Implementation Plan

The system will be implemented using Python. We will use common machine learning and data processing libraries to build and test the model.

The main tools include:

- Pandas and NumPy for data cleaning and preprocessing
- Scikit-learn for building the Random Forest model
- XGBoost as an optional advanced model
- Matplotlib or Seaborn for basic result visualization

We will use the CICIDS2017 dataset to train and evaluate the model. The system will run in a local development environment on a standard computer. This project focuses on offline analysis, not real-time deployment.

The final system will:

- Take network traffic data as input
- Classify it as benign or malicious
- Compute a risk score
- Rank alerts based on that score

The main constraint is limited computational resources and the use of a public dataset instead of live network traffic.

6 Methodology and Evaluation Plan

The project aims to create an AI based system to prioritize network intrusion detection system alerts using machine learning techniques. It will be tested using publicly available cybersecurity datasets and performance metrics.

6.1 Experimental Setup

Experiments will be executed on a computer or virtual machine that supports large-scale machine learning training operations.

Hardware and Software Environment:

Machine learning algorithms will be implemented using the following Python libraries:

- **Scikit-learn** <https://scikit-learn.org/>
- **XGBoost Framework** <https://xgboost.readthedocs.io/>

Data Processing

1. Data preprocessing and cleaning
2. Feature normalization
3. Handling missing values (NaNs)
4. Class imbalance issues (Normal Network Traffic vs Malicious Network Traffic imbalance within the dataset)
5. Model training and Hyper-parameter tuning
6. Alert Risk Score Computation
7. Performance Evaluation/Visualization

6.2 Datasets or Traffic Sources

The main data source for the project is the CIC-IDS2017 dataset provided by the Canadian Institute for Cybersecurity. The data is comprised of network traffic flow data with labels for normal traffic and different types of cyber attacks. Only the CSV files containing the data for the machine learning feature data will be used for the experiments.

No collection of network traffic is planned for the project.

6.3 Evaluation Metrics

The performance of the proposed system will be assessed based on various classification and ranking measures.

Classification Performance Metrics

- **Accuracy:** Overall accuracy of the prediction
- **Precision:** Proportion of predicted attacks that are actually attacks
- **Recall:** Ability to detect actual attacks
- **F1-score:** Harmonic mean of precision and recall
- **ROC-AUC:** Ability to distinguish between classes

Since intrusion detection systems are sensitive to false alarms, our focus will be on minimizing the false positive rates. The above metrics will be calculated on the baseline ML models, not the model that calculates the risk score.

Operational Performance Metrics

- **Alert ranking effectiveness:** Top K retrieval metrics such as Precision@K, Recall@K. Among the top K ranked alerts, how many are real attacks? What percentage of attacks are captured within the top K alerts?
- **Detection latency:** Feasibility of real-time detection using ML based techniques. Our dataset uses an offline dataset. We will need to calculate how long it takes for our model to output a risk score given an alert.

6.4 Baselines and Comparisons

1. **FIFO:** Alerts are investigated in first-in, first-out order. This is a naive baseline for processing alerts.
2. **Threshold:** Alerts labeled “High-Priority” if attack prediction probability exceeds a fixed threshold (For example, 0.5).
3. **Logistic-Regression:** Baseline ML model for comparison against the Random Forest/XGBoost models. Used to determine whether our nonlinear ensemble method improves performance in detection accuracy.
4. **Probability Score vs Risk Score:** Alerts ranked by predicted value without risk weighting. Baseline evaluation to test whether risk based assessment is better than raw probability score for ranking alerts.

7 Expected Contributions

The project aims to create an AI based system to prioritize network intrusion detection system alerts using machine learning techniques. The primary technical contribution is designing and implementing a pipeline that processes network traffic data, predicts the probability that the network data is malicious, computes a risk score, and ranks the security alerts for analyst review.

In terms of system contribution, the project integrates feature pre-processing, an ensemble machine learning model, and a risk score layer. Attack detection will be performed using a Random Forest Classifier and an XGBoost classifier. Logistic Regression will be used as the baseline model for performance evaluation with the ensemble method. The risk score mechanism will combine predicted attacked probability with attack severity to generate a prioritized alert ranking. The system will also support evaluation of alert prioritization effectiveness using ranking based metrics (Precision@K and Recall@K).

Empirically, this project does a quantitative analysis of intrusion detection performance across multiple machine learning models using the CIC-IDS2017 dataset from the Canadian Institute of Cybersecurity. Detection effectiveness will be evaluated with Accuracy, Precision, Recall, F1-score, and ROC-AUC. Since the goal of the system is to rank alerts based on severity, ranking effectiveness will be measured through Precision@K and Recall@K metrics. Detection Latency will also be analyzed to evaluate the feasibility of using the system in real-time.

Analytically, we aim to investigate the trade off between model complexity and detection performance. We will examine whether an ensemble model significantly outperforms a linear classification model and

whether risk weighting improves alert prioritization compared to probability only rankings. The results should provide insight into machine learning model use for intrusion detection and the value of AI based alert prioritization.

8 Risks and Contingency Plan

The project involves multiple components including data preprocessing, machine learning models, risk score design, and ranking evaluation. Risk have been identified along with mitigation strategies to ensure the successful completion of the project.

One primary risk of the project is model under performance and underfitting due to the class imbalance in the dataset. If the machine learning models Random Forest and XGBoost fail to predict well, mitigation strategies include hyper-parameter tuning, cross-validation for the dataset, class weighting adjustments, and feature selection. If performance of the model is still an issue, evaluation of a single model system may be used to maintain the alert prioritization aspect of the project.

There is also a potential risk due to dataset limitations. The CIC-IDS2017 dataset is from 2017 and may not fully represent modern attack patterns and the dataset could potentially require a significant effort to preprocess. Alternative public datasets may be used a supplementary or replacement dataset in these cases. Additionally, synthetic alert stream can be generated to simulate prioritization scenarios.

Tool compatibility risk would include library version conflicts, memory constraints due to the large dataset, and the computational overhead of training the models. These risk will be mitigated using stable ML frameworks such as scikit-learn and XGBoost, working with sampled subsets of data during training, and performing experiments using cloud computing/university resources if necessary.

We should also consider scope risk. Classification, risk scoring, and ranking evaluation could potentially expand beyond the intended timeline. If necessary, we will prioritize the core components of the project: (1) Ensemble ML Model, (2) Probability vs risk adjusted ranking comparison, and (3) Precision@K evaluation. More advanced weighting strategies and extensive hyperparameter tuning can also be reduced if there are time constraints.

Overall, due to the modularity of the project components can be simplified without compromising the overall objective of the project, to evaluate machine learning based alert prioritization.

9 Work Plan and Timeline

Week	Planned Activities
7	Dataset acquisition, environment setup, and initial data cleaning/EDA.
8	Data preprocessing, normalization, class imbalance handling, and feature preparation.
9	Implementation of baseline models (FIFO, threshold, Logistic Regression).
10	Development and tuning of ensemble models (Random Forest, XGBoost).
11	Design and implementation of risk scoring framework.
12	Ranking evaluation using Precision@ K and Recall@ K .
13	Latency measurement and operational feasibility analysis.
14	Evaluation, error analysis, and result consolidation.
15	Final report writing, visualization preparation, and presentation.

10 Ethical Considerations

The project will be conducted entirely with a publicly available and legal dataset, CIC-IDS2017. No live network traffic will be captured, no real systems scanned, probed, or exploited. Experimentation will be performed in a controlled local environment.

The project does not involve any penetration testing or interaction with external networks. There will be no unauthorized scanning, intrusion, or attack simulations. The work being done is strictly limited to offline data analysis.

In terms of privacy, the dataset being used consists of anonymized network traffic collected in a research setting. It does not contain any personally identifiable information linked to any individuals. This data is being used solely for academic purposes.

The overall intent of this project is defense: improve intrusion detection prioritization and reduce analyst workload. Although ML techniques in cybersecurity can potentially be misused, the focus of the project is on a purely defensive application. No new vulnerabilities are being discovered or tested.

11 Conclusion

Network intrusion detection systems can generate a large volume of alerts, which can be false positives or low priority events. This can create significant challenges for security analysts who must manually assess alerts with limited time and limited resources. Additionally, inefficient prioritization increases the risk that more critical threats are overlooked or addressed too late. Therefore it is essential to improve alert prioritization using better algorithms and automated methods for cybersecurity operations.

This project aims to create an AI based framework for prioritizing alerts using machine learning models. Using the CIC-IDS2017 dataset, the system will implement and evaluate baseline and ensemble models, including Logistic Regression, Random Forest, and XGBoost. Beyond traditional metrics, like accuracy and precision, the project introduces a risk score to rank alerts using TopK metrics. This allows evaluation not only for detection performance, but for how effectively the system prioritizes the most critical alerts. There are also operational considerations, including false positive reduction and detection latency, which will be analyzed to assess the feasibility of the system.

The expected impact of the project is to demonstrate using machine learning techniques to significantly improve alert prioritization compared to more naive baseline models as well as probability only rankings for alerts. Using the risk scoring mechanism, the proposed system aims to align automated detection with real world cybersecurity operations.

References

- [1] F. Jalalvand, M. B. Chhetri, S. Nepal, and C. Paris, “Alert prioritisation in security operations centres: A systematic survey on criteria and methods,” *ACM Computing Surveys*, vol. 57, no. 2, pp. 1–36, Nov 2024.
- [2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [3] M. Turcotte, F. Labrèche, and S.-O. Paquette, “Automated alert classification and triage (aact): An intelligent system for the prioritisation of cybersecurity alerts,” *arXiv preprint arXiv:2505.09843*, 2025.
- [4] B. Gelman, S. Taoufiq, T. Vörös, and K. Berlin, “That escalated quickly: An ml framework for alert prioritization,” *arXiv preprint arXiv:2302.06648*, Feb 2023. [Online]. Available: <https://arxiv.org/abs/2302.06648>
- [5] M. A. M. Farzaan, M. C. Ghanem, A. El-Hajjar, and D. N. Ratnayake, “Ai-powered system for an efficient and effective cyber incidents detection and response in cloud environments,” *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 3, pp. 623–643, 2025.